

Harness的工程演进和落地实践

从系统能力到企业级生产闭环

演讲嘉宾：艾长青

目录

CONTENT

- 01 范式演进：从 Prompt 到 Harness
- 02 落地挑战：工程难题拆解
- 03 实践原则：AI 时代的研发方式

01

范式演进：从Prompt到Harness

为什么企业需要的不仅仅是 **PROMPT** 技巧，更需要 **HARNESS**
——运行框架

AI 的演进：从交互到执行

从单轮问答到多步执行，每一阶段的跃迁都源于技术能力的突破

01

替代搜索

直接给答案
典型形态是 Chatbot

02

辅助工作

帮我写、改、解释
典型形态是 Copilot

03

解决任务

拆解、执行、交付
典型形态是 Agent。

04

承担完整流程

跨工具持续推进
典型形态是 Agent System。

技术演进的本质是能力边界的扩展：从生成文本，到调用工具，到管理状态，每一步都在重新定义 AI 能做什么

从 Prompt 到 Harness 的四阶段演进

01 Prompt

表达意图



定义角色、任务
格式与约束

02 Context

注入知识



连接文档
数据与代码资产

03 Tool / MCP

获得行动



调用 API、工具
与执行环境

04 Harness

形成闭环



整合状态、编排验证与
治理

从"人写好提示词", 到系统让 Agent 可执行、可验证、可治理。

Harness才是大模型时代的真正胜负手

同一模型，仅优化外围“运行脚手架，性能即可成倍跃升。范式从“换模型”转向 Harness 工程。

① 基准测评实证 · 同模型换脚手架，分数成倍跃升

SWE-bench Particula

42% → 78%

仅改脚手架即翻倍；弱模型+好脚手架反超旗舰

Terminal-Bench 2.0 LangChain

52.8% → 66.5%

固定 GPT-5.2-Codex，使用更好的harness 排名 Top30→Top5

BrowseComp 智能体搜索 Anthropic

84.0% → 86.8%

固定 Opus 4.6，加装多智能体 Harness 后分数再提升

行业综述 《Agent Harness Survey》

代码编辑任务成功率能飙升 20%+

辅以 Harness 防错机制

② 落地实战案例 · 构建 2D 复古游戏

对比维度	单代理 Solo	完整 Harness
------	----------	------------

执行时间	约 20 分钟	约 6 小时
------	---------	--------

API 成本	\$9	\$200
--------	-----	-------

任务拆解	1 句提示直接做	16 功能 / 10 Sprint
------	----------	-------------------

核心可玩性	× 实体无响应，游戏是坏的	✓ 可移动实体、真正可玩
-------	---------------	--------------

Harness: 从能力调用到生产闭环

Model = 发动机, Harness = 整车系统周边; 只有模型, Agent 无法稳定进入生产



02

落地挑战：工程难题拆解

真正的难点，不在模型够不够强，而在工程闭环是否成立

一个业务目标，如何变成可执行结果

当智能体从"生成建议"进入"负责交付"，工程问题会全面暴露。

统计

70%~95% 的 Agent 在生产环境中遭遇失败 (Fiddler AI)

Demo中表现完美，但进入到多步骤、多系统的业务流程，任务状态、上下文可信度、工具安全、结果验证每一项都会暴露问题

失败是大模型不够聪明？

01

目标定义

澄清目标、拆解任务、定义验收标准

02

信息准备

收集数据、对齐口径、权限过滤

03

执行推进

工具调用、状态追踪、人工协同

04

验证交付

质量验证、结果复用、经验沉淀

每个环节都是 Agent 落地的工程难题，哪个环节没做好都无法达成业务目标

问题一：任务链路失控

任务中断导致无限重试，不仅进度丢失无法完成任务，还烧掉高额账单

导致的后果

Agent处理长周期文档任务中断后从头开始，进度丢失，影响项目进度

为什么会这样？

上下文窗口不够大？

大模型是无状态的，一旦会话中断，上下文即刻丢失

解法：引入外部状态 或 存档机制

状态定义机制

结构化定义当前进度、版本与上下文边界

存储解绑策略

将会话状态从模型上下文中剥离至外部文件

历史信息管理

一个任务多次用户提问；一次提问多次大模型调用 -> 均作为历史信息喂给LLM

摘要总结

上下文过长 -> 历史信息生成摘要 -> 摘要中保存当前的状态和处理进度

必须像玩游戏一样自动存档，确保中断后能从断点恢复，而不是重开一局。

问题二：上下文失真

企业知识不是越多越好，而是要可导航、可追溯、可授权、可评估。

导致的后果

2024年因AI幻觉和错误决策，
企业损失近700亿

案例：使用过期的设备维护手册
导致生产线停机

为什么会这样？

RAG的错误认知
喂给AI的数据过多，可能发生
“中间丢失”

解法：建立严格的知识边界与压缩策略

知识边界划定

明确定义哪些数据可用、过期或需
权限隔离

来源追踪

强制要求关键结论必须附带数据来
源与时效

输入质量校验

对进入上下文的信息做结构化校验
(格式、完整性、一致性)，拒绝
低质量输入进入推理链。

压缩策略

通过策略压缩、语义与全局重建压
缩策略确保可以长时间调用大模型
而不超最大上下文

企业级应用准确性不可妥协，必须将“自由发散”变成“基于受控知识库的开卷考试”。

问题三：工具调用风险

工具不仅是能力延伸，还是责任扩展——行动与验证脱节，风险随之暴露。

导致的后果

PocketOS公司使用Agent排查问题，因错误调用API引发删库跑路引发的灾难

为什么会这样？

行动与验证脱节，赋予极高API权限却没有建立责任边界

目标明确，但没有考虑误操作后果

解法：建立严格的工具治理闭环与权限沙箱

📋 能力与风险分级

明确定义工具能力边界，严格划分风险等级

🧪 沙箱机制

在隔离的沙箱环境中验证高危工具调用结果

🛡️ 人工审批

高风险操作强制拦截，遵循Deny→Ask→Allow

📄 执行审计追踪

完整记录工具调用参数与结果，建立责任链

提示词随时可能被覆盖，必须在执行层建立物理隔离，确保模型发疯也无法破坏。

问题四：结果难信任

Agent最大的风险不是“不会做”，而是“看起来很有把握地错了”。

导致的后果

金融场景，Agent生成审计报告错误引用法规条款，导致监管罚款

为什么会这样？

Agent不懂装懂

验证机制缺失，人类核对认知负担太重

解法：构建多层次的自动化验证流水线

✅ 确定性规则校验

在输出前自动校验数值、格式与合规性规则

🔍 权威事实比对

将关键事实与企业内部权威数据源进行交叉比对

🎭 独立评估打分

引入独立模型或 LLM-as-Judge 对输出质量打分

🚫 门禁拦截

未通过自动化证据链验证的结果，严禁进入下游

把质量检查变成机器自动执行的卡点，提供自动化证据链，人类才能放心信任。

问题五：生产不可观测

没有追踪的智能体，就像没有日志、监控和链路追踪的分布式系统。

导致的后果

Agent自动化做营销方案，运行一夜发现没有任何产出

原因：API格式变化导致不断重试

为什么会这样？

Agent往往是逻辑上的失败，传统监控失效；
缺乏针对推理过程和工具调用的追踪机制

解法：建立完整轨迹的细粒度可观测体系

推理路径追踪

细粒度记录 Agent 的每步推理决策与分岔点

工具调用串联

将分散的工具调用输入输出串联为完整调用链

实时状态看板

实时展示系统运行状态，异常行为触发自动干预

逻辑级根因复盘

基于完整轨迹记录，实现逻辑级错误定位与优化

不可观测就不可调试。必须看清 Agent 的思维过程，才能快速定位问题根源。

问题六：经验难复用

个人会用智能体，不代表组织能规模化用好智能体。

导致的后果

资深员工精心设计对话，生成完美的数据分析报告，当员工离开导致经验流失

为什么会这样？

知识和交互技巧存在于个人脑中，没有结构化沉淀

解法：引入 Skills（技能）机制进行团队级资产沉淀

从个人实践到团队资产



Skill的定义

如何定义好的 Skill：具体可验证、避免矛盾



Skill 粒度设计

控制粒度：太大不够通用，太小导致数量爆炸



团队资产升级

如何推广：从个人经验沉淀升级为组织复用资产



Skill 安全管控

防范安全风险：如何防止恶意指令或 Skill 投毒

为什么这么做：AI 生产力必须从个人写提示词的技巧，升级为组织可复用的标准化资产。

问题七：协作规模失控

多 Agent 互相覆盖文件，协调者彻底迷失在海量代码细节中

真实后果

多个 Agent 协同开发软件项目失败，相互之间存在任务冲突，结果相互覆盖，导致项目失败

为什么会这样？

上下文污染与职责边界模糊，每个 Agent 都试图掌握全局

解法：分工明确，严格摘要回传

协作原则



职责边界隔离

严格区分 主Agent 与 子Agent，禁止越权操作
只有主Agent负责与用户交互 + 任务的规划和拆解



严格摘要回传

子任务（子Agent）仅回传结构化摘要，绝不允许倾灌全文



异步协作

主Agent与子Agent异步交互，定义标准的通讯协议
子Agent并发执行

必须确保主Agent的上下文始终保持纯净，始终聚焦于全局计划的推进。

问题八：知识不沉淀

每次新项目都要重新“培训”AI，效率极低且经常遗忘旧规则

真实后果

每次启动新项目，Agent像个新人，都要重复纠正之前项目中犯过的错误

为什么会这样？

记忆架构缺失：当前 LLM 每次推理都是全新的，缺乏长期记忆架构

解法：构建分层与带宽感知的企业级记忆架构

顶层稳定知识 始终加载：构建命令、数据口径等企业稳定规范

中层主题文件 按需读取：根据当前任务类型动态加载相关决策

底层海量记录 仅检索不加载：历史记录按需检索，不占上下文

静默知识蒸馏 后台整合：静默将对话经验蒸馏为持久项目知识

没有记忆沉淀就没有复利，必须让 AI 记住纠错和规范，成为团队成长的成员。

实施路线与框架：先建小闭环，再做平台化

大模型落地切忌好高骛远，必须通过小闭环跑通工程机制，再横向扩展能力。

01

找准切入点

切入点原则：高频、风险可控、验收明确，避免一上来就挑战核心业务核心系统。

02

定义边界

明确验收标准：写清交付物格式、质量及格线与不可越过的安全风险边界。

03

受控执行

接入最少必要工具：建立工具登记册，在沙箱演练，设置高危操作的人工审批门禁。

04

闭环验证

建立评测基线：沉淀成功与失败样例，设置发布门禁，让输出结果有客观证据支撑。

05

能力复用

沉淀为组织资产：将跑通的上下文、工具组合与治理策略封装为可复用的 Skill。

Harness 落地挑战：新能力和旧体系的矛盾（例如：旧有系统的改造成本、团队习惯改变的阻力）

先在单个场景完成从意图到执行的工程闭环，验证后再做平台化，避免在不成熟的基座上堆叠复杂度

03

实践原则：AI 时代的研发方式

HARNESS 不仅是技术框架，它也在倒逼团队工作方式的改变。

认知转变与 Label 范式重构

从"写代码"到"与文档对话", 背后是数据飞轮的根本重构。

01 认知转变: 交互范式的演进

传统开发

人的角色: 代码的直接编写者

人 → 代码

AI 协同阶段

人的角色: 代码的审阅者

人 ↔ 模型

Vibe Coding 时代

人的角色: 意图的表达者

人 → 文档 → 模型 → 代码

02 数据飞轮的重构: Label 范式的根本转变

补全 / NES 时代

现象

用户手动修改 AI 生成内容

本质

人机协同纠错 (AI 辅助, 人兜底)



Agent 时代

现象

人只提需求, 代码无人工修改痕迹

本质

意图与执行的分层分工

最高质量的数据不再是"被修改的代码", 而是"被澄清的意图与验收标准".

验证粒度跃迁与测试前置

大模型生成代码极快，验证方式必须随之升级。

01 验证粒度的跃迁：从 Code Review 到 Function Review

AI 一秒钟生成 500 行代码，人类花 10 分钟逐行 Review → 未能提效，反而增加认知负担

关注点转移：放弃“代码实现细节”，转向“功能黑盒行为”的验证

02 测试前置：评测数据必须独立于功能开发

评测数据与功能开发共享上下文 = 考生自己出卷自己答题

错误做法

- ✗ 开发完成后再补测试
- ✗ 同一模型会话生成功能与评测数据
- ✗ 测试通过即认为功能正确

正确做法

- ✓ 需求阶段同步设计测试用例
- ✓ 评测数据独立于功能开发上下文
- ✓ 评测标准开发前锁定，不随实现变化

不要 Review 每一行代码，用独立测试用例约束行为——好的评测体系是质量的护城河。

Vibe Coding 的反噬与文档即契约

初期越快，后期越可能失控——文档是唯一的抓手。

01 警惕 Vibe Coding 的反噬：隐形技术债的爆发

初期（蜜月期）

开发极快，需求迅速落地，AI 独立完成大量功能

后期（深水区）

面对大量“非自己编写”的代码，排查几乎不可能，甚至不如从头开始

没有 Harness 约束的 Vibe Coding，是在透支未来的可维护性。

02 文档即契约：让大模型维护文档

上下文来源

模型依赖文档理解项目现状

团队契约

文档记录“为什么这样做”的决策逻辑

同步流程

代码改动 → 触发更新 → 模型基于模板更新 → 人工 Review

格式稳定

Skill 模板保障文档格式一致

Harness 下沉为 AgentOS

Harness 下沉为 AgentOS，是 AI 提效的迫切需求：让人彻底从工具操作中解放，只专注于目标表达

上层：AI 原生应用层（人类唯一的工作界面）

人只需表达目标与意图，不再关心工具调用、代码实现与执行细节——彻底解放人力

中间层：AgentOS（Harness 下沉后的智能体操作系统）

自动承接意图解析、任务拆解、Agent 协作调度、上下文记忆与工具安全控制
现在 Vibe Coding 需要人工干预的所有环节，未来由 AgentOS 自动完成

下层：传统操作系统与底层算力硬件

Windows / macOS / Linux · 文件系统 · CPU / GPU 计算资源

工作模式的跃迁：从 Vibe Coding（人写文档 → 模型写代码）到 AgentOS（人表达目标 → 智能体全自动交付），人类从执行者彻底升级为决策者。

THANKS

演讲嘉宾：艾长青

企业真正要建设的，不是更聪明的模型，而是可执行、可验证、可治理的智能体运行时。